



Angular – 21.X.X



Productivity Meets Scalability

Effortless Angular Testing

What is Testing?

Testing or Software testing is the process of evaluating a software product to ensure it functions correctly, meets all specified requirements, and is free of defects or bugs.

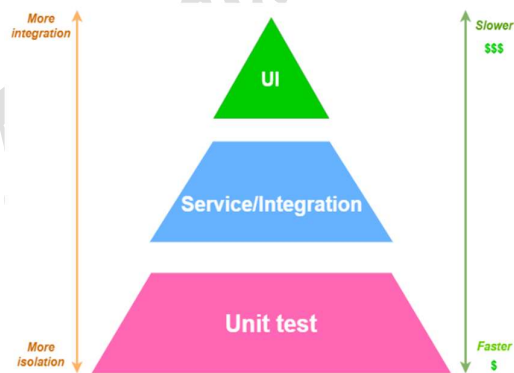


It involves executing the software, often using automated or manual tools, to identify issues, improve its quality, reliability, and user satisfaction, and to provide objective information about its state before it's released to users.

Testing is the process of executing a program or system with the intent of finding defects and verifying that requirements are met.

Most engineering teams rely on the **Testing Pyramid** to structure their automation:

1. **Base (Unit Tests):** The majority of your tests. They ensure the foundations are strong, providing instant feedback while coding.
2. **Middle (Integration Tests):** A moderate amount of tests to bridge the gaps and make sure your internal logic actually speaks properly to databases and external APIs.
3. **Top (E2E Tests):** A minimal, selective suite reserved for high-stakes, mission-critical business flows (like signing up or paying money) where a failure breaks the entire product.



Testing Terminology

Before going to learn Testing with practical examples and walkthroughs, understand meaning of few most common terms of testing:

Term	Meaning
Test	Code that verifies another piece of code
Test Case	One specific scenario being tested
Test Suite	Collection of related tests
Unit Test	Tests a small isolated unit
Integration Test	Tests multiple units working together
E2E Test	Tests application from user's perspective
Assertion	Verification that expected == actual
Fixture	Controlled environment/data used by test
Mock	Fake replacement for dependency
Stub	Provides predefined responses
Spy	Observes whether/how a method was called



Angular – 21.X.X



Productivity Meets Scalability

Fake	Simplified working implementation
Test Double	General term for mock/stub/spy/fake
Setup	Code executed before tests
Teardown	Cleanup after tests
Coverage	How much code is exercised by tests
Regression Test	Ensures existing functionality doesn't break
Positive Test	Valid input/expected scenario
Negative Test	Invalid/error scenario
Edge Case	Boundary/unusual scenario
AAA	Arrange → Act → Assert

Real-World Example: An E-Commerce Checkout

To see how they work together, consider building an online shopping cart:

- **Unit Test:** You test the isolated function `calculateTax(price, state)`. You pass it a price of ₹100 and a state of Telangana, validating that it returns exactly ₹115.75 without touching an external database or network.
- **Integration Test:** You test the checkout system's communication with external boundaries. You trigger the `processOrder` system to verify that your backend properly saves the order to a local test database and makes a successful network request to a mock payment processor API.
- **End-to-End Test:** A script boots up a real browser, simulates a customer landing on the homepage, clicks "Add to Cart," fills out physical shipping info, inputs simulated credit card details, hits "Purchase," and verifies that a "Thank You" screen appears alongside an automated email confirmation.

What is unit testing?

Unit testing checks the smallest testable pieces (units) of code—typically functions, classes, components, or services—in isolation from external dependencies. Benefits of unit testing are early bug detection, safer refactoring, design feedback, documentation of expected behavior.

It acts as the foundation of the modern testing pyramid, serving as the first line of defense against bugs before code moves to integration and end-to-end testing phases. Unit testing is a mandatory process, by which we can identify all the defects related to security, safety, and reliability. Unit testing helps us to improve the quality of our code.

What is the purpose that unit tests serve?

What happens when you're working on an application and you want to make a change to existing code? Will this break the application? How do you know?

If you do make a change to a function, and there are tests already written that are good enough to cover that change, you should be able to make the change with confidence. They should also serve as a form of documentation for your application, somewhere you can go to read and figure out what the app is intended to do when you can't figure this out from the code.



Angular – 21.X.X

Productivity Meets Scalability



Testing is the process of verifying that software behaves according to its requirements. For example, a typescript function:

```
function add(a: number, b: number): number {  
  return a + b;  
}
```

We expect:

`add(10, 20) → 30`

A test verifies this expectation automatically. Almost every unit test can be explained using **AAA**:

Arrange → Act → Assert

```
it('should add two numbers', () => {  
  // Arrange  
  const a = 10;  
  const b = 20;  
  // Act  
  const result = add(a, b);  
  // Assert  
  expect(result).toBe(30);  
});
```

Here:

- **Arrange** prepares the test.
- **Act** executes the functionality.
- **Assert** verifies the result.

The description above will be the foundation for whatever we are going to learn. Before Angular, let's understand that testing is not an Angular concept, it is fundamentally a software-development concept.

Unit testing in JavaScript

It tends to the practice of isolating the smallest pieces of your application in JavaScript—typically individual functions or methods—and writing automated scripts to verify that they behave correctly under various conditions. Instead of opening a browser and manually clicking through a UI, unit tests execute your logic with defined inputs and programmatically assert that the output matches your expectations.

Popular JavaScript Testing Frameworks

While you can write tests using pure JavaScript conditionals, developers almost exclusively use modern testing frameworks to handle test execution, assertions, and reporting.



Angular – 21.X.X



Productivity Meets Scalability

- **Jest:** The most popular and comprehensive testing framework. Developed by Meta, it works out of the box with zero configuration for most projects and includes a test runner, assertion library, and mocking tools.
- **Vitest:** A blazing-fast, modern test runner built on top of Vite. It shares a highly compatible API with Jest and is highly popular in modern frontend setups (React, Vue, Svelte).
- **Mocha & Chai:** A highly flexible combination. Mocha acts as the test runner, while Chai provides the assertion logic.
- **Jasmine and Karma:**

Walkthrough Unit Testing for JavaScript function using Jest:

1. Make sure Node.JS is installed with npm.
2. Create an application folder: **sampleProject**
3. Open it in Visual Studio Code / or you can work with command prompt itself
4. In Visual Studio Code start a new terminal and fire:

```
npm init -y
npm install --save-dev jest
```

5. Wrote to ...\\package.json:

```
{
  "name": "sampleProject",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \\\"Error: no test specified\\\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

6. Update the scripts object in your package.json file to run Jest:

```
"scripts": {
  "test": "jest"
},
```

7. Create a Source File (math.js) and write a simple functions you want to test:

```
// math.js
function add(a, b) {
  if (typeof a !== "number" || typeof b !== "number") {
    throw new Error("Only numbers are allowed");
  }
  return a + b;
}
module.exports = { add };
```



Angular – 21.X.X

Productivity Meets Scalability



8. Write the Unit Test File (math.test.js)
9. Jest automatically scans for files ending in .test.js or .spec.js.

```
// math.test.js
const { add } = require("../math");
describe("Math Operations Suite", () => {
  // Positive Test Case
  test("should correctly add two numbers (Happy Path)", () => {
    {
      // Arrange
      const num1 = 5;
      const num2 = 10;
      // Act
      const result = add(num1, num2);
      // Assert
      expect(result).toBe(15);
    });
    // Negative Test Case
    test("should throw an error if inputs are strings", () => {
      // Act & Assert combined for error handling testing
      expect(() => add("5", 10)).toThrow("Only numbers are
allowed");
    });
  });
});
```

10. Run the Tests, Execute the script in your terminal

npm test

11. Jest will output a structured report showing your test suites, individual test statuses (green checkmarks or red errors), and execution times.

12. **Pass Result:** Terminal highlights the successful test in green showing

5 + 10 = 15 below ↓

```
D:\ngprac\Day43>npm test
> day43@1.0.0 test
> jest

PASS ./math.test.js
Math Operations Suite
  ✓ should correctly add two numbers (Happy Path) (2 ms)
  ✓ should throw an error if inputs are strings (3 ms)

Test Suites: 1 passed, 1 total
Tests: 2 passed, 2 total
Snapshots: 0 total
Time: 4.947 s
Ran all test suites.
```

13. **Fail Result:** Terminal highlights the mismatch in red, showing a message like →

Expected: 15

Received: 50.

```
D:\ngprac\Day43>npm test
> day43@1.0.0 test
> jest

FAIL ./math.test.js
Math Operations Suite
  ✕ should correctly add two numbers (Happy Path) (4 ms)
  ✓ should throw an error if inputs are strings (2 ms)

• Math Operations Suite › should correctly add two numbers (Happy Path)

expect(received).toBe(expected) // Object.is equality

Expected: 15
Received: 50

10 |     const result = add(num1, num2);
11 |     // Assert
12 |     expect(result).toBe(15);
    |                     ^
13 |   });
14 |   // Negative Test Case
15 |   test("should throw an error if inputs are strings", () => {

at Object.toBe (math.test.js:12:20)

Test Suites: 1 failed, 1 total
Tests: 1 failed, 1 passed, 2 total
Snapshots: 0 total
Time: 0.397 s, estimated 3 s
Ran all test suites.
```